

CHAPTER 11

11 - Prisma 기초 (쇼핑몰 시작 — SQLite → PostgreSQL)

Node.js · Express · NestJS 강의 · 2026

10장의 posts는 메모리 배열이라 서버를 재시작하면 사라졌다. 이 장에서 NestJS 파트의 본 프로젝트 ****쇼핑몰(shop)****을 시작한다. 상품·분류 CRUD를 만들고 **Prisma + SQLite**로 진짜 저장한 다음, 마지막에 `datasource` provider 한 줄을 바꿔 **PostgreSQL**로 옮긴다. 엔드포인트도, 모듈 구조도, 쿼리 코드도 그대로인 채 저장소만 갈아탄다 — ORM을 쓰는 이유를 직접 확인한다.

여기서 만드는 **shop 프로젝트 하나**를 16장까지 계속 키운다. 12장에서 판매자·분류 관계, 13장 인증, 14장 캐싱... 한 장씩 쌓아 17장에서 완성형 쇼핑몰이 된다.

아직 관계는 없다. `Product.categoryId` 는 분류의 id를 담는 보통 숫자 칸일 뿐, 외래키가 아니다. 진짜 관계 (`@relation`, `include`)는 12장에서 건다.

이 장에서 배우는 것

- **Prisma 스키마** — 모델·자료형·속성 정의, 마이그레이션
- **PrismaService** — DB 커넥션을 NestJS 생명주기에 거는 법
- **모듈 간 DI** — `ProductsService` 가 `CategoriesService` 를 주입받아 "분류가 있는지" 확인
- **SQLite → PostgreSQL 전환** — provider 교체와 마이그레이션 리셋
- **페이지네이션** — `skip` / `take`

0. 프로젝트 생성 — nest new shop (과정 전체에서 한 번만)

```
nest new shop --skip-git --package-manager npm
cd shop
rm src/app.controller.ts src/app.controller.spec.ts src/app.service.ts # 기본 생성물 정리

# 기능별 모듈 + 서비스 + 컨트롤러 (--no-spec: 테스트 파일은 이 과정에선 생략)
nest g module categories
nest g service categories --no-spec
nest g controller categories --no-spec

nest g module products
nest g service products --no-spec
nest g controller products --no-spec
```

10장처럼 `nest g resource categories --no-spec` 으로 한 번에 받아도 된다(생성된 `entities/` 폴더는 이 과정에선 안 쓴다). DTO는 직접 채운다.

1. 패키지 설치

```
npm i class-validator class-transformer dotenv
npm i @prisma/client@6
npm i -D prisma@6
```

버전 기준 — 이 과정의 코드는 **NestJS 11 + Prisma 6**에서 검증했다. package.json을 배포하지 않으므로 각자 설치하는데, Prisma는 메이저(6)를 붙여 설치한다.

2. Prisma 초기화 — 먼저 SQLite로

```
npx prisma init --datasource-provider sqlite # prisma/schema.prisma + .env 생성
```

SQLite는 파일 하나가 곧 데이터베이스라 서버 설치 없이 바로 시작할 수 있다. 생성된 `.env` 에 `PORT` 를 추가한다.

```
DATABASE_URL="file:./dev.db"
PORT=3000
```

3. 스키마 작성 + 마이그레이션

`prisma/schema.prisma` 에 모델 두 개를 적는다. 자료형·속성의 뜻은 아래 "Prisma 스키마 — 자료형과 모델 기초" 절에 정리해 뒀다.

```
model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique
}

model Product {
  id          Int    @id @default(autoincrement())
  name        String
  description String
  price       Int
  stock       Int    @default(0)
  categoryId  Int    // 아직 @relation 없음 - 분류 id를 숫자로만 보관
  createdAt   DateTime @default(now())
}
```

```
npx prisma migrate dev --name init # prisma/dev.db(SQLite) 파일과 테이블 생성
```

DB 커넥션을 NestJS에 연결할 Prisma 모듈/서비스도 만든다.

```
nest g module prisma
nest g service prisma --no-spec
```

`PrismaService` 는 `PrismaClient` 를 상속해 `OnModuleInit` 에서 `$connect()` 한다. `PrismaModule` 은 `@Global` 로 두어 어느 모듈에서나 주입받게 한다.

4. 코드 작성 순서

순 서	파일	내용
1	src/prisma/prisma.service.ts · prisma.module.ts	PrismaClient 상속 + \$connect() · 모듈 은 @Global + exports
2	src/categories/dto/create-category.dto.ts	name 검증(@IsString , @NotEmpty)
3	src/categories/categories.service.ts	분류 생성·조회(Prisma), findOne 은 없으면 404
4	src/categories/categories.controller.ts	POST /categories · GET /categories · GET /categories/:id
5	src/categories/categories.module.ts	CategoriesService 를 exports — products가 쓴다
6	src/products/dto/create-product.dto.ts · update-product.dto.ts	본문 검증(@Min(0) 등) · Update는 모든 필드 @IsOptional
7	src/products/dto/query-product.dto.ts	page · limit 선택 필드(@IsOptional , @Type(() => Number))
8	src/products/products.service.ts	상품 CRUD(Prisma) + CategoriesService 주입(분류 존재 확인) + findMany 에 skip / take
9	src/products/products.controller.ts · products.module.ts	5개 라우트 · CategoriesModule 을 imports
10	src/main.ts	import "dotenv/config" + 전역 ValidationPipe({ whitelist: true, transform: true })

각 파일의 코드와 설명은 해설서 11장에 있다.

5. 실행 + 동작 테스트 — SQLite

```
npm run start:dev      # → http://localhost:3000
```

```

BASE=http://localhost:3000

# 분류 생성 (이름은 @unique → 중복이면 409)
curl -s -X POST $BASE/categories -H "Content-Type: application/json" -d '{"name":"전자기"}'

# 상품 등록 - 없는 분류면 404 (ProductsService가 CategoriesService에 확인 = 모듈 간 DI)
curl -s -X POST $BASE/products -H "Content-Type: application/json" \
  -d '{"name":"무선 이어폰","description":"노이즈 캔슬링","price":890000,"stock":10,"categoryId":1}'

# 목록 - 페이지네이션
curl -s "$BASE/products?page=1&limit=10"

# 수정 / 삭제
curl -s -X PATCH $BASE/products/1 -H "Content-Type: application/json" -d '{"price":80000}'

```

여기서 10장과의 차이를 확인하자 — 서버를 재시작해도 데이터가 남아 있다. `prisma/dev.db` 파일에 저장됐기 때문이다.

6. PostgreSQL로 갈아타기

SQLite는 시작이 가볍지만 동시 접속이 많은 운영 환경에는 한계가 있다. 같은 코드를 그대로 둔 채 저장소만 PostgreSQL로 옮긴다.

6-1. 준비 — DBeaver로 접속 확인 + nest_shop 만들기

이 과정은 각자 맥에 설치된 PostgreSQL(dmg 설치본)을 쓴다. 앞서 DBeaver로 접속해 본 그 서버다.

1. DBeaver를 열고 **Database Navigator**에서 기존 PostgreSQL 연결을 더블클릭한다. 트리가 펼쳐지면 서버가 떠 있는 것이다. (연결 에러가 나면 PostgreSQL을 먼저 시작한다.)
2. 연결을 우클릭 → **Edit Connection(연결 편집)** → **Main** 탭에서 적어 둔다: **Host**(보통 `localhost`) · **Port**(보통 `5432`) · **Username**(보통 `postgres`). 비밀번호는 화면에 보이지 않는다 — 설치 때 정한 값을 쓴다.
3. 연결을 펼쳐 **Databases** 폴더를 우클릭 → **Create New Database** → 이름 `nest_shop` → **OK**. (SQL로 하려면 SQL Editor에서 `CREATE DATABASE nest_shop;`)

이 DB 하나를 16장까지 계속 쓴다.

6-2. provider 한 줄 + 접속 URL 교체

```
prisma/schema.prisma :
```

```
datasource db {
  provider = "postgresql" // ← "sqlite" 에서 변경
  url      = env("DATABASE_URL")
}
```

`.env` — `<사용자>` · `<비밀번호>` 는 6-1에서 확인한 본인 값으로:

```
DATABASE_URL="postgresql://<사용자>:<비밀번호>@localhost:5432/nest_shop?schema=public"
PORT=3000
```

6-3. 마이그레이션 리셋 — 딱 한 번 ⚠

지금까지의 `prisma/migrations` 는 **SQLite** 방언으로 생성된 **SQL**이다. `provider`만 바꾸고 `migrate`를 돌리면 Prisma가 `migration_lock.toml` 의 `provider` 불일치 에러를 낸다. 그래서 이력을 비우고 PostgreSQL 기준으로 다시 시작한다.

```
rm -rf prisma/migrations prisma/dev.db # SQLite 이력과 DB 파일 삭제
npx prisma migrate dev --name init      # nest_shop 에 테이블 생성
```

스키마(`schema.prisma`)가 단일 진실이라 이력을 지워도 앓는 건 없다 — 새 `init` 이 현재 스키마 전체를 PostgreSQL 방언으로 다시 만든다. 단, 데이터는 따라오지 않는다(SQLite의 상품은 `dev.db` 와 함께 사라진다). 리셋은 `provider`를 바꾸는 이번 한 번뿐이고, 12장부터는 `migrate dev --name <이름>` 으로 이력이 쌓인다.

6-4. 다시 실행 — 같은 curl이 그대로 돈다

```
npm run start:dev
```

5번의 curl을 그대로 다시 실행해 보자. 서비스 코드는 한 글자도 안 바꿨는데 이제 PostgreSQL에 저장된다.

```
# 정말 들어갔는지 DBeaver에서 확인 - SQL Editor(상단 DB 선택을 nest_shop 으로):
SELECT id, name, price FROM "Product";    -- 테이블명이 대문자라 따옴표 필수
```

Prisma 스키마 — 자료형과 모델 기초

`prisma/schema.prisma` 에 데이터 모양을 적고, `prisma migrate dev` 로 DB에 표를 만든다. NestJS 서비스에서는 생성된 `PrismaClient` (`PrismaService`)로 CRUD한다.

스키마 파일 구조

```
generator client {
  provider = "prisma-client-js" // TypeScript용 클라이언트 생성
}

datasource db {
  provider = "sqlite" // 이 장 도입: sqlite / 이 장 끝: postgresql
  url      = env("DATABASE_URL") // .env의 접속 문자열
}

model Category { ... } // model 하나 = DB 테이블 하나
model Product { ... }
```

블록	역할
generator	npx prisma generate 시 @prisma/client 타입-메서드 생성
datasource	어떤 DB에 붙을지 (provider + DATABASE_URL)
model	테이블·컬럼 정의. 모델명 Product → 코드에서는 prisma.product (소문자)

스키마를 정리할 때: npx prisma format

이 장에서 쓰는 자료형

Prisma 타입	의미	예시
Int	정수	id , price , stock , categoryId
String	문자열	name , description
DateTime	날짜-시간	createdAt
Boolean	참/거짓	(이 장에는 없음)

금액은 **Int** 로 둔다. price 를 Float 로 두면 소수 연산 오차가 날 수 있어서, 원 단위 정수(89000)로 저장한다.

자주 쓰는 필드 속성(어노테이션)

속성	의미	예시
@id	기본 키(PK)	Category.id , Product.id
@default(autoincrement())	INSERT 시 id 자동 증가	id
@default(0)	값을 안 넣으면 기본값 0	Product.stock
@default(now())	INSERT 시 현재 시각	Product.createdAt
@unique	중복 불가(유니크 제약)	Category.name

? 를 붙이면 **선택(nullable)** 필드다. 이 장의 모델은 전부 필수(? 없음)다.

categoryId 는 분류의 id 를 가리키지만, 이 장에서는 **외래키(@relation)**를 아직 안 건다. 앱 코드 (CategoriesService.findOne)로 존재 여부를 확인한다. 진짜 관계는 12장.

스키마 → DB 반영 (마이그레이션)

```
npx prisma migrate dev --name init # schema → SQL → DB 적용, migrations/ 에 기록
npx prisma generate               # migrate 시 보통 자동 실행됨
```

명령어	역할
prisma migrate dev	개발 중 스키마 변경을 DB에 적용 + 마이그레이션 파일 생성
prisma generate	schema.prisma 기준으로 PrismaClient 타입·API 재생성
prisma studio	브라우저에서 테이블 데이터 확인 (npx prisma studio)
prisma format	schema.prisma 들여쓰기·정렬

모델을 바꿀 때마다 migrate dev 를 다시 실행한다.

서비스에서 쓰는 기본 CRUD

PrismaService 가 PrismaClient 를 상속하므로, 서비스에서 this.prisma.product , this.prisma.category 로 접근한다.

작업	Prisma API	이 장의 코드
전체 조회	<code>findMany()</code>	<code>prisma.product.findMany({ orderBy: { id: "desc" } })</code>
단건 조회	<code>findUnique({ where })</code>	<code>findUnique({ where: { id } })</code>
생성	<code>create({ data })</code>	<code>create({ data: dto })</code>
수정	<code>update({ where, data })</code>	<code>update({ where: { id }, data: dto })</code>
삭제	<code>delete({ where })</code>	<code>delete({ where: { id } })</code>
페이지네이션	<code>skip / take</code>	<code>findMany({ skip: (page-1)*limit, take: limit })</code>

DTO(`CreateProductDto`) 필드명과 모델 필드명이 같으면 `create({ data: dto })` 처럼 객체를 그대로 넘길 수 있다.

모델명 ↔ 코드에서의 이름

schema.prisma	Prisma Client
<code>model Category</code>	<code>prisma.category</code>
<code>model Product</code>	<code>prisma.product</code>

TypeScript 타입도 생성된다: `Product`, `Category` (`@prisma/client` 에서 import 가능).

이 장에서 다루지 않는 것 (뒤 장 예고)

주제	언제
<code>@relation</code> , <code>include</code>	12장 — 판매자·분류 관계
<code>enum</code>	13장 — 역할(Role)
트랜잭션(<code>\$transaction</code>)	15장 — 주문

핵심 개념

모듈 간 DI — 한 서비스가 다른 서비스를 부른다

상품을 만들 때 "그 분류가 실제로 있는가"를 확인해야 한다. 이 판단은 분류를 아는 `CategoriesService` 의 몫이다. 그래서 `ProductsService` 가 `CategoriesService` 를 주입받아 물어본다.

```
constructor(private readonly categoriesService: CategoriesService) {}

create(dto: CreateProductDto) {
  this.categoriesService.findOne(dto.categoryId); // 없으면 404
  // ...
}
```

이게 되려면 `CategoriesModule` 이 `CategoriesService` 를 `exports` 하고, `ProductsModule` 이 `CategoriesModule` 을 `imports` 해야 한다. 모듈은 "무엇을 내보내고 무엇을 들여올지"로 협력한다. 10장에서 본 "컨트롤러 ← 서비스" 주입의 모듈 버전이다.

PrismaService — 커넥션을 생명주기에 건다

```
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  async onModuleInit() {
    await this.$connect(); // 모듈이 뜰 때 DB에 연결
  }
}
```

ORM의 가치 — DB를 바꿔도 코드는 안 바뀐다

이 장에서 직접 확인했다. 서비스 코드(create·findAll·findOne·update·remove)는 그대로인 채 `datasource` 만 SQLite에서 PostgreSQL로 바뀌었다. Prisma가 같은 쿼리 API를 DB별 SQL로 번역해 주기 때문이다. 단, 마이그레이션 이력은 DB 방언으로 기록되므로 provider를 바꿀 때 한 번 리셋했다(6-3).

핵심 파일

파일	역할
<code>prisma/schema.prisma</code>	모델 정의(Category·Product) + DB 접속
<code>src/prisma/prisma.service.ts</code>	<code>PrismaClient</code> + 생명주기 연결
<code>src/prisma/prisma.module.ts</code>	<code>@Global</code> — 전역에서 주입
<code>src/categories/categories.service.ts</code>	분류 저장 + <code>findOne</code> (없으면 404)
<code>src/products/products.service.ts</code>	상품 CRUD + <code>CategoriesService</code> 주입 + 페이지네이션
<code>src/main.ts</code>	<code>dotenv</code> + 전역 <code>ValidationPipe</code>

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면, 수업 저장소의 `11/` 폴더(완성 코드, **PostgreSQL** 전환이 끝난 상태)로 이 장의 끝에서 다시 시작할 수 있다.

```
cd 11
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_11) - 자동 생성이 안 되면
DBBeaver로 먼저 만든다
npm run start:dev
```

다음 장

- **12 - Prisma 관계:** 판매자(User)를 더해 상품과 1:N로 잇고, 단일 분류(categoryId)를 여러 분류(M:N)로 승격한다.